

Listex 11

João Pedro Machado Silva, BV3032477

Ex3:

- **Bucket Sort**

O Bucket Sort é um algoritmo de ordenação baseado na técnica de distribuição. A ideia principal é dividir o conjunto de dados em um número de baldes (buckets), distribuir os elementos desses dados entre os baldes, e então ordenar os elementos de cada balde individualmente. Depois disso, os baldes ordenados são concatenados para formar o array final ordenado.

Passos:

1. **Criar Buckets:** Divida o intervalo de dados em baldes (buckets) com base no valor máximo dos dados.
2. **Distribuir Dados:** Coloque cada elemento no balde correspondente.
3. **Ordenar Buckets:** Ordene cada balde individualmente (usando qualquer método de ordenação).
4. **Concatenar:** Combine todos os baldes ordenados de volta ao array original.

```
void bucketSort(float array[], int n) {  
    int i, j;  
    int bucketCount = n;  
    float bucket[bucketCount][n];  
    int bucketSizes[bucketCount];
```

```

// Inicializa os tamanhos dos buckets como 0
for (i = 0; i < bucketCount; i++) {
    bucketSizes[i] = 0;
}

// Distribui os elementos nos buckets
for (i = 0; i < n; i++) {
    int bucketIndex = n * array[i]; // Supondo que os elementos estão
entre 0 e 1
    bucket[bucketIndex][bucketSizes[bucketIndex]++] = array[i];
}

// Ordena os elementos em cada bucket
for (i = 0; i < bucketCount; i++) {
    for (j = 1; j < bucketSizes[i]; j++) {
        float key = bucket[i][j];
        int k = j - 1;
        while (k >= 0 && bucket[i][k] > key) {
            bucket[i][k + 1] = bucket[i][k];
            k--;
        }
        bucket[i][k + 1] = key;
    }
}

// Concatena os buckets no array original
int index = 0;
for (i = 0; i < bucketCount; i++) {
    for (j = 0; j < bucketSizes[i]; j++) {
        array[index++] = bucket[i][j];
    }
}
}

```

Complexidade:

No melhor e médio caso, os elementos não caem no mesmo bucket, então não se tem a ordenação interna de complexidade n^2 como no pior caso.

Melhor caso: $O(n+k)$, onde n é o número de elementos e k é o número de buckets.

Pior caso: $O(n^2)$, quando todos os elementos caem no mesmo bucket e o algoritmo de ordenação interna é $O(n^2)$.

Caso Médio: $O(n + k)$, onde n é o número de elementos e k é o número de buckets.

- **Counting Sort**

O Counting Sort é um algoritmo de ordenação não comparativa que funciona contando o número de ocorrências de cada valor distinto no array. Esses contadores são então usados para posicionar os elementos na posição correta no array ordenado.

Passos:

1. **Encontrar Máximo e Mínimo:** Identifique o valor máximo e mínimo no array.
2. **Contar Ocorrências:** Crie um array de contagem para armazenar a quantidade de vezes que cada valor aparece.
3. **Calcular Posições:** Modifique o array de contagem para representar as posições acumuladas.

4. **Construir Array Ordenado:** Preencha o array de saída com base no array de contagem.
5. **Copiar Resultado:** Copie o array ordenado de volta para o array original

```
void countingSort(int array[], int n) {
    int i;
    int max = array[0];
    int min = array[0];

    // Encontrando o valor máximo e mínimo no array
    for (i = 1; i < n; i++) {
        if (array[i] > max) {
            max = array[i];
        }
        if (array[i] < min) {
            min = array[i];
        }
    }

    int range = max - min + 1;
    int *count = (int *)malloc(range * sizeof(int));
    int *output = (int *)malloc(n * sizeof(int));

    // Inicializa o array de contagem com 0
    for (i = 0; i < range; i++) {
        count[i] = 0;
    }

    // Contagem das ocorrências de cada elemento
    for (i = 0; i < n; i++) {
        count[array[i] - min]++;
    }

    // Atualizando o array de contagem para armazenar posições
```

```

for (i = 1; i < range; i++) {
    count[i] += count[i - 1];
}

// Construindo o array de saída
for (i = n - 1; i >= 0; i--) {
    output[count[array[i] - min] - 1] = array[i];
    count[array[i] - min]--;
}

// Copiando o array de saída para o array original
for (i = 0; i < n; i++) {
    array[i] = output[i];
}

// Liberando memória alocada
free(count);
free(output);
}

```

Complexidade:

Claramente, a complexidade é $O(n+k)$ para todos os casos, pois os fors serão executados independentemente.

Melhor caso: $O(n+k)$, onde n é o número de elementos e k é o valor máximo no array.

Pior caso: $O(n+k)$.

Caso Médio: $O(n + k)$, onde n é o número de elementos e k é o número de buckets.